

Performance Analysis: Custom 4-Layer DNN vs VGGNet

1. Setup and Quantitative Results

Both networks were trained on the CIFAR-10 dataset for 100 epochs. The training progression was evaluated using cross-entropy loss and classification accuracy.

Table 1: Performance metrics comparison on CIFAR-10 after 100 epochs.

Metric / Feature	My_DNN (Custom)	VGGNet (Baseline)
Architectural Depth	4 Conv Layers	16 Layers (VGG)
Parameters (approx.)	$\sim 2.5 \times 10^5$	$\sim 1.5 \times 10^7$
Final Training Loss	0.6032	0.2438
Final Training Accuracy	79.45%	91.81%
Final Test Accuracy	71.52%	85.48%
Generalization Gap (Δ)	7.93%	6.33%

2. Performance Discussion and Architectural Trade-Offs

A. Representational Capacity and Feature Hierarchy

The primary driver behind the performance gap between My_DNN (71.52%) and VGGNet (85.48%) is model capacity and depth:

- **VGGNet:** utilizes stacked 3×3 convolutional layers across 16 stages. This depth enables the network to learn an intricate feature hierarchy—progressing from low-level edges and textures in early layers to abstract, high-level object components in deeper layers.
- **In my DNN,** the limited depth (4 convolutional layers) restricts the effective receptive field before reaching the spatial reduction stages. Consequently, the network struggles to distinguish complex, overlapping visual features inherent to spatial datasets like CIFAR-10.
- **Parameter Scale:** VGGNet is much larger than my model, with roughly 15 million parameters compared to just 250,000. Because it has so much more memory and learning capacity, VGGNet can easily spot fine details in images that my smaller model misses.

B. Training Dynamics and Optimization

My_DNN demonstrated exceptionally stable training dynamics, despite its small number of parameters:

- The inclusion of a residual skip connection prevented vanishing gradients during early epochs, allowing rapid convergence during the first 30 epochs.
- **Internal Covariate Shift Reduction:** Integrating BatchNorm2d after every convolutional layer maintained normalized activation distributions across batches, enabling stable learning at higher learning rates ($\eta = 0.1$).

C. Generalization and Regularization

The model achieved 79.45% training accuracy and 71.52% test accuracy—a difference of only 7.93%. This narrow gap shows that Dropout ($p = 0.3$) and Global Average Pooling successfully kept the model from overfitting, allowing it to perform reliably on new test data.